Table 11-39. *`TwigFilter` class instantiation techniques*

`new TwigFilter('filter_name', 'func_name')`	Registers the PHP named function
`new TwigFilter('filter_name', 'Class::method')`	Registers static `method` defined in `Class`
`new TwigFilter('filter_name', ['Class', 'method'])`	Registers static `method` defined in `Class`
`new TwigFilter('filter_name', [$this, 'method'])`	Registers `method` defined of object `$this`

Controlling Behavior

Given the nature of filters, there is some flexibility around escaping and safety. By default, your filter extension's output is subject to autoescaping just like the built-in ones, which might not be ideal if your filter is already handling escaping. Conversely, you might require that input is already escaped before your own code gets to work on it, rather than working with raw input. Twig also provides mechanisms for injecting the template context (the set of variables accessible from that template) and even the entire `Twig\TwigEnvironment` instance, should it be required. These options are controlled by passing an associative array of booleans as the third argument constructor and result in extra parameters for the filter callable:

```
// index.php
$filter = new \Twig\TwigFilter('addcat', function (array $context,
$string) {
    return $string . "... and Jess the cat. {$context['catNoise']}.";
}, ['needs_context' => true]);
$twig->addFilter($filter);

print $twig->render('add-cat.html.twig', [
    'catNoise' => 'Purr purr'
]);
```